PROJECT AND TEAM INFORMATION

Project Title

QuestForge : A Free-World 2D RPG Integrating Data Structures, Algorithms, and Object-Oriented Programming Concepts

Student / Team Information

Team Name: *Team #*	*SegFault Studios* *DSCPP-III-2026-T227*
Team Member 1 (Team Lead)	*Diwaker, Anushka – 2510011707* *anushkadiwaker9d@gmail.com* 
Team Member 2	*Rawat, Shobhit – 2510013357* *shobhitrawat2150@gmail.com* 
Team Member 3	*Khanduri, Ashu – 2510012142* *ashukhanduri07@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Our project is an interactive 2D RPG game integrating data structures, algorithms, and concepts of object-oriented programming.
So why are we doing this? The first question arises: describe the problem. The problem these days is rote learning. It's how students are expected to learn concepts of data structures, how students are expected to go on LeetCode and learn algorithms in college themselves, and not make it actually a fun thing to do. It's how object-oriented programming is just considered an important part of your curriculum, but not actually made known to you, not actually made easier for you to understand.
These are the problems we're having every day, especially because we as students are going through that phase right now, where studying gets tougher by the day. We're confused about how to manage all these things together and how to not get bored, but how to make sure that you're not going to burn out doing all these three things together.
So this solution came up: we can design a game which can be interactive, which can be fun, and at the same time make you learn concepts, different concepts of data structures, about algorithms, about the concepts of object-oriented programming, on how one object can be used from the same class in so many different manners, and a lot of other techniques of object-oriented programming.

State of the Art / Current solution (1 pt)

This problem these days is solved by making our lectures a bit more interactive about learning and coding, making these concepts of loops and algorithms easier to understand, however they're not as easy as expected.
We are taught this not only in our college, but also through YouTube, many online courses these days, and websites like LeetCode and Codeforces, which are trying to be as interactive and as easy for a user to understand as possible, but a student in particular is unable to connect to it as much as it can with a game-like interface, keeping them busy.
A student has so many things to work around these days that they are just unable to hold on to and grasp so many concepts. At a point, it gets boring, and at a point, it gets frustrating for a student to learn and just rote-learn so much.

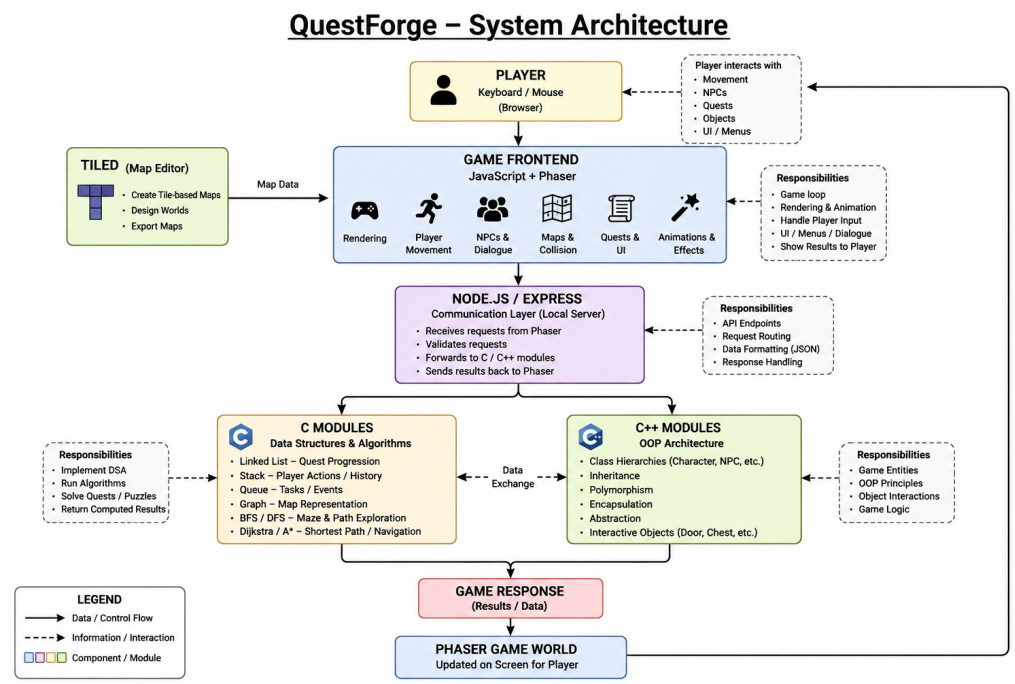
Project Goals and Milestones (2 pts)

The general goals of this project are to make sure that the user learns concepts of data structures like queues, stacks and linked lists. The user also grasps concepots of algorithms like Dijkstra, BFS, DFS, and concepts of OOPs, like polymorphism, inheritance and data abstraction.
We want to make sure that the user knows how data structures are used in day-to-day life, as well as algorithms, and make sure that the user knows that knowledge is important, but at the same time, so are human etiquettes. If you don't have the normal etiquettes of a human being required to carry you around in this world, then any level of knowledge, rather it be high or low, is not worth it or important at all. Those are the main project goals.
Some initial milestones include finalizing our game concept and all the quests that will be in the game, deciding how we are going to use all the knowledge we have in implementing this project, using all the technologies and architecture that has been provided to us by our faculty and our college to the fullest.
Some later milestones will include building the systems, building the quest systems, implementing data structures and OOP systems, and connecting the game frontend with C and C++ projects. While some final milestones being testing and preparing the final playable prototype and documentation.

Project Approach (3 pts)

We have decided that the game frontend will be made using JavaScript and Phaser. The actual game will run in a browser, and we'll use JavaScript and Phaser for things like rendering, player movement, maps, NPCs, and animations. The maps will be made using Tiled, which is a separate tile-based map editor that works with Phaser. Everything the player interacts with, basically, will be made with the help of JavaScript and Phaser. All the assets will again be based on a tile-based system.
Then we'll have data structures and algorithms. These will be implemented; for example, the quest progressions will be kept in linked lists, the player's actions and history system will be kept in a stack, and tasks and events will be in a queue. The map will first be represented as a graph, and then we have algorithms like BFS and DFS for maze and path exploration, as well as Dijkstra and A for pathfinding and navigation.*
Then we'll use an OOP architecture for, of course, making our NPCs. For example, we'll have a class called Character, and then it will have different objects where we have different NPCs. Of course, we'll have our own interactive player as well. We'll have InteractiveObject as a class, for example, and then we'll have different objects for it, like doors, chests, etc. Using our OOP concepts, we will make the entire game object-oriented. Lastly connecting everything using Node.js as a communication layer.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The main outcomes of the project will be a working 2D RPG game that we will be able to run on a browser. The RPG will have interactive quests through which users can learn concepts of data structures, algorithms, and OOPs. It will also have practical implementations of C, data structures, and OOPs concepts such as inheritance and polymorphism, while also incorporating human etiquettes and general knowledge about the world.
It will also have interactive environments with NPCs, maps, dialogues, puzzles, and mazes.
The deliverables of this project will be the prototype of the playable game, the source code that will be used, the communication layer using Node.js, all the maps and assets used, the documentation about the entire project, the system architecture, and the final presentation.

Assumptions

The main assumptions of the project will be that the required development tools are available to the team, including Node.js, C/C++ compilers, JavaScript, Phaser, Tiled, etc.
It is also assumed that the user has access to a browser capable of running the game and has basic knowledge of how a computer and keyboard controls are used.
The project also assumes that the C and C++ modules can communicate with the JavaScript and Node.js layers.

References

- *Material Provided by our faculty, all the lecture notes, referenced books.*
- *GeeksforGeeks*
- *Node.js and Express.js Documentation*
- *Phaser Documentation*